

BEDROCK ARCHITECTURE

Bedrock Target Intrinsic

C Compiler Builtins and Header Interfaces

Draft

CONTENTS

1	Scope and Status	1
2	Compiler Interface Model	1
2.1	Names and Ownership	1
2.2	Header Topology and Exposure	1
2.3	Common Compilation Rules	2
3	Public Target Interface	2
3.1	Core Builtins	2
3.2	Memory Builtins	2
3.3	Integer Builtins	3
3.4	Floating-Point Builtins	3
3.5	Approximate Transcendental Lowering Policy	6
4	System Target Interface	6
4.1	System-Register Builtins	6
4.2	Cache-Management Builtins	7
4.3	MMU Builtins	8
4.4	Processor-State Builtins	8
5	Shared Header Types	9

LIST OF TABLES

2-1	Target Intrinsic Header Families	1
3-1	Core Intrinsics	2
3-2	Memory Intrinsics	2
3-3	Integer Intrinsics	3
3-4	Floating-Point Intrinsics	3
4-1	System-Register Intrinsics	6
4-2	Cache-Management Intrinsics	7
4-3	MMU Intrinsics	8
4-4	Processor-State Intrinsics	8
5-1	Target Intrinsic Shared Types	9

1 SCOPE AND STATUS

This document defines the compiler-owned Bedrock C target interface: builtin names, implementation-reserved header wrappers, source signatures, lowering requirements, availability and privilege constraints, and compiler-visible effects. It is the contract shared by a Bedrock-aware C frontend, optimizer, backend, and the target headers.

Architectural instruction behavior remains defined by the Bedrock ISA Specification. The baseline calling convention is defined by the Bedrock C ABI. This document specifies how source programs reach target facilities; it does not duplicate their architectural or language semantics.

Interface name: bedrock-target-intrinsics

Source language: ISO C with implementation-reserved target interfaces

2 COMPILER INTERFACE MODEL

2.1 Names and Ownership

The compiler builtin namespace begins with `__builtin_bedrock_`. Each **Name** entry in this document denotes that prefix followed by the displayed name. For example, `cuid` denotes `__builtin_bedrock_cuid`. The ordinary header wrapper uses the `__bedrock_` prefix with the same name unless its entry specifies a macro spelling.

All of these identifiers are implementation-reserved. A builtin is a compiler primitive, not an external function symbol. The target headers own wrapper declarations and umbrella include topology; this document owns their ABI-visible meaning.

2.2 Header Topology and Exposure

`<bedrockintrin.h>` is the public umbrella. It includes the core, memory, integer, and floating-point families. `<bedrocksystemintrin.h>` is the system umbrella for system-register, cache, MMU, and processor-state facilities. Any family header may also be included directly.

Table 2-1. Target Intrinsic Header Families

Family	Header	Umbrella	Exposure
Cache-Management	<code><bedrockcacheintrin.h></code>	system	cache management
Core	<code><bedrockcoreintrin.h></code>	public	unprivileged core operations
Floating-Point	<code><bedrockfpintrin.h></code>	public	floating-point environment, classification, and approximate operations
Integer	<code><bedrockintegerintrin.h></code>	public	nonportable integer operations
Memory	<code><bedrockmemoryintrin.h></code>	public	user ordering and access hints
MMU	<code><bedrockmmuintrin.h></code>	system	supervisor translation operations
Processor-State	<code><bedrockstateintrin.h></code>	system	runtime state management
System-Register	<code><bedrocksysregintrin.h></code>	system	architectural system-register access

2.3 Common Compilation Rules

Family wrappers are static inline functions unless an encoded immediate or type operand requires a macro. Every builtin listed by an enabled profile must be recognized by the compiler. If a required optional feature is disabled, compilation fails rather than emitting an unresolved call.

An encoded immediate must be an integer constant expression in the stated range. Header inclusion neither suppresses nor emulates architectural privilege checks. An operation is volatile only when its entry states an externally observable architectural effect. Compiler memory effects and hardware ordering are exactly those stated by the entry; one does not imply the other.

The **C interface** column uses `result(arguments)` shorthand. Exact declarations are in the named header; `u8`, `u16`, `u32`, and `u64` abbreviate the corresponding `uintN_t` types, and `result` denotes `__bedrock_query_result_t`. A clobber means a compiler memory clobber, while a named fence also has the hardware behavior defined by the ISA.

3 PUBLIC TARGET INTERFACE

3.1 Core Builtins

Header: `<bedrockcoreintrin.h>`

Table 3-1. Core Intrinsics

Name	C interface	Lowering	Availability, constraint, and effect
<code>breakpoint</code>	<code>void(void)</code>	BKPT	Any privilege; always raises the architectural breakpoint/debug exception.
<code>cpuid</code>	<code>u64(u64)</code>	CPUID	User allowed; reads discovery state and not C memory.
<code>rdpmc</code>	<code>u64(performance_counter)</code>	RDPMC	Unprivileged; ID is a constant in 0 through 65535; zero and unavailable IDs fault with <code>INVALID_SELECTOR</code> .
<code>read_status</code>	<code>u16(void)</code>	RDSTATUS	Unprivileged; reads <code>STATUS</code> and not C memory.
<code>relax</code>	<code>void(void)</code>	RELAX	Unprivileged PAUSE-like convergent hint; retires normally and permits a finite implementation-selected delay, including zero, before subsequent execution.
<code>trace</code>	<code>void(u32)</code>	TRACE	Unprivileged; marker is a constant in 0 through 65535; emits a marker when enabled and otherwise acts as a NOP.
<code>yield</code>	<code>void(void)</code>	YIELD	Any privilege; scheduling hint, not a compiler or memory barrier.

3.2 Memory Builtins

Header: `<bedrockmemoryintrin.h>`

Table 3-2. Memory Intrinsics

Name	C interface	Lowering	Availability, constraint, and effect
address_fence	void(void)	AFENCE	Full compiler memory clobber and hardware address fence.
nontemporal_store_u16	void(u16 *,u16)	MOVNT.W	One writable 16-bit object; one hinted store, neither volatile nor a fence.
nontemporal_store_u32	void(u32 *,u32)	MOVNT.L	One writable 32-bit object; one hinted store, neither volatile nor a fence.
nontemporal_store_u64	void(u64 *,u64)	MOVNT.Q	One writable 64-bit object; one hinted store, neither volatile nor a fence.
nontemporal_store_u8	void(u8 *,u8)	MOVNT.B	One writable byte; one hinted store, neither volatile nor a fence.
read_fence	void(void)	RFENCE	Compiler and hardware read-ordering barrier.
write_fence	void(void)	WFENCE	Compiler and hardware write-ordering barrier.

3.3 Integer Builtins

Header: <bedrockintegerintrin.h>

Table 3-3. Integer Intrinsics

Name	C interface	Lowering	Availability, constraint, and effect
clmul_u16	u16(u16,u16)	CLMUL.W	Pure; low 16 bits of the carryless product.
clmul_u32	u32(u32,u32)	CLMUL.L	Pure; low 32 bits of the carryless product.
clmul_u64	u64(u64,u64)	CLMUL.Q	Pure; low 64 bits of the carryless product.
clmul_u8	u8(u8,u8)	CLMUL.B	Pure; low 8 bits of the carryless product.

3.4 Floating-Point Builtins

Header: <bedrockfpuinttrin.h>

Table 3-4. Floating-Point Intrinsics

Name	C interface	Lowering	Availability, constraint, and effect
facosa_f32	float(float)	FACOSA.S	FP and FPTRANSA required; ISA accuracy, special-value, and floating-point event contracts apply.
facosa_f64	double(double)	FACOSA.D	FP and FPTRANSA required; ISA accuracy, special-value, and floating-point event contracts apply.
fasina_f32	float(float)	FASINA.S	FP and FPTRANSA required; ISA accuracy, special-value, and floating-point event contracts apply.
fasina_f64	double(double)	FASINA.D	FP and FPTRANSA required; ISA accuracy, special-value, and floating-point event contracts apply.
fatana_f32	float(float)	FATANA.S	FP and FPTRANSA required; ISA accuracy, special-value, and floating-point event contracts apply.

Name	C interface	Lowering	Availability, constraint, and effect
fatana_f64	double(double)	FATANA.D	FP and FPTRANSA required; ISA accuracy, special-value, and floating-point event contracts apply.
fatanha_f32	float(float)	FATANHA.S	FP and FPTRANSA required; ISA accuracy, special-value, and floating-point event contracts apply.
fatanha_f64	double(double)	FATANHA.D	FP and FPTRANSA required; ISA accuracy, special-value, and floating-point event contracts apply.
fclass_f32	u16(float)	FCLASS.S	FPU required; pure ten-class one-hot classification; does not update FFLAGS.
fclass_f64	u16(double)	FCLASS.D	FPU required; pure ten-class one-hot classification; does not update FFLAGS.
fcosa_f32	float(float)	FCOSA.S	FP and FPTRANSA required; ISA accuracy, special-value, and floating-point event contracts apply.
fcosa_f64	double(double)	FCOSA.D	FP and FPTRANSA required; ISA accuracy, special-value, and floating-point event contracts apply.
fcosha_f32	float(float)	FCOSHA.S	FP and FPTRANSA required; ISA accuracy, special-value, and floating-point event contracts apply.
fcosha_f64	double(double)	FCOSHA.D	FP and FPTRANSA required; ISA accuracy, special-value, and floating-point event contracts apply.
fetoxa_f32	float(float)	FETOXA.S	FP and FPTRANSA required; ISA accuracy, special-value, and floating-point event contracts apply.
fetoxa_f64	double(double)	FETOXA.D	FP and FPTRANSA required; ISA accuracy, special-value, and floating-point event contracts apply.
fetoxm1a_f32	float(float)	FETOXM1A.S	FP and FPTRANSA required; ISA accuracy, special-value, and floating-point event contracts apply.
fetoxm1a_f64	double(double)	FETOXM1A.D	FP and FPTRANSA required; ISA accuracy, special-value, and floating-point event contracts apply.
flog10a_f32	float(float)	FLOG10A.S	FP and FPTRANSA required; ISA accuracy, special-value, and floating-point event contracts apply.
flog10a_f64	double(double)	FLOG10A.D	FP and FPTRANSA required; ISA accuracy, special-value, and floating-point event contracts apply.
flog2a_f32	float(float)	FLOG2A.S	FP and FPTRANSA required; ISA accuracy, special-value, and floating-point event contracts apply.
flog2a_f64	double(double)	FLOG2A.D	FP and FPTRANSA required; ISA accuracy, special-value, and floating-point event contracts apply.
flogna_f32	float(float)	FLOGNA.S	FP and FPTRANSA required; ISA accuracy, special-value, and floating-point event contracts apply.
flogna_f64	double(double)	FLOGNA.D	FP and FPTRANSA required; ISA accuracy, special-value, and floating-point event contracts apply.

Name	C interface	Lowering	Availability, constraint, and effect
<code>flognp1a_f32</code>	<code>float(float)</code>	<code>FLOGNP1A.S</code>	FP and FPTRANSA required; ISA accuracy, special-value, and floating-point event contracts apply.
<code>flognp1a_f64</code>	<code>double(double)</code>	<code>FLOGNP1A.D</code>	FP and FPTRANSA required; ISA accuracy, special-value, and floating-point event contracts apply.
<code>fsina_f32</code>	<code>float(float)</code>	<code>FSINA.S</code>	FP and FPTRANSA required; ISA accuracy, special-value, and floating-point event contracts apply.
<code>fsina_f64</code>	<code>double(double)</code>	<code>FSINA.D</code>	FP and FPTRANSA required; ISA accuracy, special-value, and floating-point event contracts apply.
<code>fsincosa_f32</code>	<code>void(float,float *,float *)</code>	<code>FSINCOSA.S</code>	FP and FPTRANSA required; writes sine through the first result pointer and cosine through the second; ISA accuracy, special-value, and floating-point event contracts apply to both results.
<code>fsincosa_f64</code>	<code>void(double,double *,double *)</code>	<code>FSINCOSA.D</code>	FP and FPTRANSA required; writes sine through the first result pointer and cosine through the second; ISA accuracy, special-value, and floating-point event contracts apply to both results.
<code>fsinha_f32</code>	<code>float(float)</code>	<code>FSINHA.S</code>	FP and FPTRANSA required; ISA accuracy, special-value, and floating-point event contracts apply.
<code>fsinha_f64</code>	<code>double(double)</code>	<code>FSINHA.D</code>	FP and FPTRANSA required; ISA accuracy, special-value, and floating-point event contracts apply.
<code>ftana_f32</code>	<code>float(float)</code>	<code>FTANA.S</code>	FP and FPTRANSA required; ISA accuracy, special-value, and floating-point event contracts apply.
<code>ftana_f64</code>	<code>double(double)</code>	<code>FTANA.D</code>	FP and FPTRANSA required; ISA accuracy, special-value, and floating-point event contracts apply.
<code>ftanha_f32</code>	<code>float(float)</code>	<code>FTANHA.S</code>	FP and FPTRANSA required; ISA accuracy, special-value, and floating-point event contracts apply.
<code>ftanha_f64</code>	<code>double(double)</code>	<code>FTANHA.D</code>	FP and FPTRANSA required; ISA accuracy, special-value, and floating-point event contracts apply.
<code>ftentoxa_f32</code>	<code>float(float)</code>	<code>FTENTOXA.S</code>	FP and FPTRANSA required; ISA accuracy, special-value, and floating-point event contracts apply.
<code>ftentoxa_f64</code>	<code>double(double)</code>	<code>FTENTOXA.D</code>	FP and FPTRANSA required; ISA accuracy, special-value, and floating-point event contracts apply.
<code>ftwotoxa_f32</code>	<code>float(float)</code>	<code>FTWOTOXA.S</code>	FP and FPTRANSA required; ISA accuracy, special-value, and floating-point event contracts apply.
<code>ftwotoxa_f64</code>	<code>double(double)</code>	<code>FTWOTOXA.D</code>	FP and FPTRANSA required; ISA accuracy, special-value, and floating-point event contracts apply.
<code>read_fflags</code>	<code>u16(void)</code>	<code>RDFFLAGS</code>	Unprivileged; reads accrued floating-point exception flags.

Name	C interface	Lowering	Availability, constraint, and effect
<code>read_fstatus</code>	<code>u16(void)</code>	<code>RDFSTATUS</code>	Unprivileged; access to the floating-point environment.
<code>write_fflags</code>	<code>void(u16)</code>	<code>WRFFLAGS</code>	Unprivileged; reserved bits follow the ISA; writes accrued flags.
<code>write_fstatus</code>	<code>void(u16)</code>	<code>WRFSTATUS</code>	Unprivileged; reserved bits follow the ISA; changes the floating-point environment.

3.5 Approximate Transcendental Lowering Policy

A strict language-library operation shall not be lowered to an FPTRANSA instruction. A compiler may select FPTRANSA only when the source uses a separately defined approximate interface or an active compiler mode permits approximate mathematics. The selected instruction remains subject to its ISA accuracy, special-value, and floating-point exception-condition contract.

4 SYSTEM TARGET INTERFACE

The compiler emits the requested architectural operation; runtime privilege and policy checks remain active.

4.1 System-Register Builtins

Header: `<bedrocksysregisterintrin.h>`

Table 4-1. System-Register Intrinsics

Name	C interface	Lowering	Availability, constraint, and effect
<code>read_code_segment</code>	<code>u64(void)</code>	<code>RDSEG CS</code>	Unprivileged; reads the current CS image through the dedicated source-only encoding.
<code>read_control_register</code>	<code>u64(control_register)</code>	<code>RDCR</code>	Supervisor; selector is a constant in 0 through 65535 naming a defined register.
<code>read_segment_register</code>	<code>u64(segment_register)</code>	<code>RDSEG</code>	Unprivileged; selector is a compile-time <code>__bedrock_segment_register_t</code> .
<code>write_control_register</code>	<code>void(control_register, u64)</code>	<code>WRCR</code>	Supervisor; defined constant selector required; writes the register and clobbers memory.
<code>write_segment_register</code>	<code>void(segment_register, u64)</code>	<code>WRSEG</code>	Unprivileged; validates the SREG image, changes address interpretation, and fully clobbers memory.
<code>write_status</code>	<code>void(u16)</code>	<code>WRSTATUS</code>	Supervisor; reserved and privilege-controlled bits follow the ISA; compiler-visible side effect.

The system-register header defines `__bedrock_control_register_t` constants for every published control-register selector. The constants are names for the architectural selector values; they do not weaken the privilege, reserved-bit, or per-register validation performed by `RDCR` and `WRCR`.

It also defines the pure value macros `__BEDROCK_SEGMENT_IMAGE`, `__BEDROCK_SEGMENT_IMAGE_FOR_BASE`, and `__BEDROCK_SEGMENT_DISABLED`. They construct 64-bit segment images without reading or writing architectural state.

`__BEDROCK_SEGMENT_IMAGE(base_page,exponent,mantissa, bounds_only)` places the low 52, 5, and 6 bits of the first three operands in their architectural fields and normalizes `bounds_only` to zero or one. Each operand is evaluated once. A valid enabled image uses an exponent from 0 through 31 and a mantissa from 1 through 63. The macro only packs fields; it does not validate the resulting base, span, limit, or address canonicity.

`__BEDROCK_SEGMENT_IMAGE_FOR_BASE(base,...)` encodes $\lfloor (\text{uint64_t})\text{base}/4096 \rfloor$ in the base-page field. It discards the low 12 bits, so a misaligned operand names the containing page and does not preserve the original byte address. The macro imposes no alignment precondition and does not adjust a later offset to compensate for the discarded bits. `__BEDROCK_SEGMENT_DISABLED` is the canonical all-zero disabled image.

4.2 Cache-Management Builtins

Header: `<bedrockcacheintrin.h>`

All cache range operations evaluate the address once and obtain the maintenance granule from CPUID `CACHE_TOPOLOGY`. A zero length emits no cache-maintenance instruction. A nonzero, non-wrapping byte range selects every maintenance block that intersects the range and lowers to one block instruction per selected block. Each instruction translates its own block address. If a later instruction faults, blocks completed by earlier instructions remain completed.

For a nonzero length, the mathematical half-open range $[\text{address}, \text{address} + \text{length})$ shall lie within the 64-bit address space. Otherwise the behavior of the call is undefined and no architectural cache-maintenance sequence is required. An implementation may diagnose a provably wrapping constant range. A valid range end is calculated mathematically, not modulo 2^{64} .

Table 4-2. Cache-Management Intrinsics

Name	C interface	Lowering	Availability, constraint, and effect
<code>flush_dcache</code>	<code>void(void *,size_t)</code>	<code>FLSHDCACHE</code>	Supervisor; applies <code>FLSHDCACHE</code> to every CPUID maintenance block intersecting the range and fully clobbers memory.
<code>invalidate_dcache</code>	<code>void(void *,size_t)</code>	<code>INVDCACHE</code>	Supervisor; applies <code>INVDCACHE</code> to every CPUID maintenance block intersecting the range and fully clobbers memory.
<code>invalidate_icache</code>	<code>void(void *,size_t)</code>	<code>INVICACHE</code>	Supervisor; applies local <code>INVICACHE</code> to every CPUID maintenance block intersecting the range, serializes instruction fetch, and fully clobbers memory.
<code>sync_cache</code>	<code>void(void *,size_t)</code>	<code>SYNCCACHE</code>	Supervisor; applies <code>SYNCCACHE</code> to every CPUID maintenance block intersecting the range, serializes instruction fetch, and fully clobbers memory.

Name	C interface	Lowering	Availability, constraint, and effect
writeback_dcache	void(void *,size_t)	WRBKDCACHE	Supervisor; applies WRBKDCACHE to every CPUID maintenance block intersecting the range and fully clobbers memory.

4.3 MMU Builtins

Header: <bedrockmmuintrin.h>

Table 4-3. MMU Intrinsics

Name	C interface	Lowering	Availability, constraint, and effect
invalidate_asid	void(u16)	INVASID	Supervisor; ASID is a constant in 0 through 65535 and the operation fully clobbers memory.
invalidate_page	void(const void *)	INVPAGE	Supervisor; argument supplies a linear address and invalidates any matching cached leaf mapping containing it; the operation fully clobbers memory.
invalidate_tlb	void(void)	INVTLB	Supervisor; translation-state side effect and full memory clobber.
page_table_query	query_result(u32,u64)	PTQUERY	Supervisor; level is an integer constant expression in 1 through 5; wrapper atomically returns value and FLAGS; page walk without access to the queried C object.
switch_page_table	void(u64)	SWPT	Supervisor; valid page-table image required; changes address space and fully clobbers memory.
switch_page_table_asid	void(u64,u16)	SWPTA	Supervisor; valid image and low 16-bit ASID; changes address space and fully clobbers memory.
virtual_to_physical	query_result(u64)	VTOP	Supervisor; wrapper returns value and operation-defined FLAGS captured atomically; page walk without access to the translated C object.

4.4 Processor-State Builtins

Header: <bedrockstateintrin.h>

Table 4-4. Processor-State Intrinsics

Name	C interface	Lowering	Availability, constraint, and effect
<code>restore_processor_state</code>	<code>void(const void *)</code>	RESTORE	Readable, feature-valid, 4096-byte-aligned area; reads the complete selected processor-state image; may define R0--R15, FLAGS, valid GS registers, and extension state; fully clobbers memory and prevents TLS or segmented memory accesses from moving across a restored GS addressing context.
<code>save_processor_state</code>	<code>void(void *)</code>	SAVE	Writable, CPUID-sized, 4096-byte-aligned area; writes architectural state and fully clobbers memory.

Both operations are unprivileged, but restoration of STATUS remains privilege-restricted. The back-end must model every restored register and the full memory and addressing-context clobber explicitly. It must not treat restore as an opaque call or as only a memory effect.

5 SHARED HEADER TYPES

Table 5-1. Target Intrinsic Shared Types

Type	ABI contract
<code>__bedrock_performance_counter_t</code>	Architectural performance-counter selectors.
<code>__bedrock_query_result_t</code>	Value and architectural flags returned by MMU queries.
<code>__bedrock_control_register_t</code>	Architectural control-register selectors.
<code>__bedrock_segment_register_t</code>	Source-selectable architectural segment registers.